08

DEEP DIVE 08 · మీరు రోజూ వాడేదాన్ని లోపలి నుంచి కట్టడం

Design a HashMap

> PayPal · Walmart · Flipkart · Uber · Goldman Sachs · Microsoft · Amazon – ఇది ఒక data structure ప్రశ్న లా కనిపిస్తుంది, కానీ interviewers దీన్ని ఒక **design** ప్రశ్నగా అడుగుతారు.

మీరు `Map`, `dict`, `HashMap` ని రోజూ వాడతారు. ఇక్కడ దాన్ని **సున్నా నుంచి** కడతాం – మరియు మూడుసార్లు విరగ్గొడతాం. మొదటి విరుపు lo ఒక విలువ నిశ్శబ్దంగా ఇంకొకదాన్ని **తినేస్తుంది**, మరియు అది మన మొదటి ఉదాహరణలోనే జరుగుతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి సంఖ్యా నిజంగా కొలిచినదే** – timings, chain పొడవులు, bucket పంపిణీ, అన్నీ.

> **ఇది DATA STRUCTURE ప్రశ్న కాదు, DESIGN ప్రశ్న**
>
> LeetCode lo "Design HashMap" ఒక సులభమైన problem – chaining రాసి పంపేయొచ్చు. Interview lo అది **మొదటి పది నిమిషాలు** మాత్రమే.
>
> అసలు ప్రశ్నలు తర్వాత వస్తాయి: **Load factor 0.75 ఎందుకు, 0.9 కాదు? Capacity రెట్టింపు ఎందుకు, +1 కాదు? చెడ్డ hash function ఏమి చేస్తుంది? Key ని మార్చితే? Chaining నా open addressing నా?**
>
> ఇవన్నీ **trade-off** ప్రశ్నలు, మరియు ఈ doc వాటి గురించే. ప్రతి జవాబుని **కొలిచి** చూపిస్తాను.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. HashMap ఏమి వాగ్దానం చేస్తుంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: వరుస వెతుకులాట

3. Step – ఒక array, ఒక loop
4. **మొదటి విరుపు** – 10 ms నుంచి 78 ms కి

Part 3 – రెండో విరుపు: ఢీకొన్న keys

5. Step – Key ని ఒక చోటిగా మార్చడం
6. **రెండో విరుపు** – `city` వచ్చి `name` ని తినేసింది
7. Step – Chaining

Part 4 – మూడో విరుపు: పొడవైపోతున్న గొలుసులు

8. **మూడో విరుపు** – 1,122 పొడవు గల ఒక గొలుసు
9. Step – Resize మరియు rehash
10. Load factor 0.75 ఎందుకు · రెట్టింపు ఎందుకు

Part 5 – Hash function యొక్క నాణ్యత

11. చెడ్డ hash – 256 buckets, 4 మాత్రమే వాడబడ్డాయి
12. మారే key – ఒక entry ని శాశ్వతంగా పోగొట్టుకోవడం

Part 6 – పూర్తి system

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 7 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – open addressing, thread safety, treeify
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. HashMap ఏమి వాగ్దానం చేస్తుంది

HashMap ఒక్క వాగ్దానం చేస్తుంది, మరియు అది ఆశ్చర్యకరమైనది:

> **"ఎన్ని items ఉన్నా, ఒక key ని వెతకడానికి ఒకే సమయం పడుతుంది."**

పది items ఉన్నా, పది లక్షలు ఉన్నా – `get(key)` ఒకే వేగం. దీన్నే **O(1)** అంటారు.

ఇది ఎలా సాధ్యం? ఒక ఉపమానం: మీ దగ్గర పది లక్షల పుస్తకాలు ఉన్న ఒక గ్రంథాలయం ఉంది.

- **వరుసగా వెతకడం** – మొదటి అరలో మొదలుపెట్టి ఒక్కొక్కటిగా చూడటం. పది లక్షల పుస్తకాలు = పది లక్షల చూపులు.
- **Hash** – పుస్తకం పేరు నుంచి *అర సంఖ్య* ని **లెక్కించడం**. "Ramayana" → అర 4,821. నేరుగా అక్కడికి వెళ్ళడం. **ఒక్క చూపు.**

ఆ "పేరు నుంచి అర సంఖ్య ని లెక్కించడం" – అదే **hash function**. ఇదే మొత్తం ఆలోచన.

కానీ ఇక్కడ మూడు కష్టాలు దాగి ఉన్నాయి, మరియు ఈ doc అవే:

"ఒక్క చూపు" అనే వాగ్దానం · మూడు చోట్ల విరుగుతుంది

hash(key) % size → నేరుగా ఆ చోటికి · ఏది తప్పు కావచ్చు?

1 · ఢీకొట్టడం
రెండు keys, ఒకే అర సంఖ్య
(పది లక్షల పుస్తకాలు,
వెయ్యి అరలు – తప్పదు)
→ §6 · ఒకటి మాయమవుతుంది

2 · నిండిపోవడం
అరలు 16, పుస్తకాలు 16,000
ప్రతి అరలో వెయ్యి పుస్తకాలు
→ మళ్ళీ వరుస వెతుకులాట
→ §8 · O(1) పోయింది

3 · చెడ్డ లెక్క
Hash అన్నిటినీ ఒకే
అరకి పంపితే?
వెయ్యి అరలు ఖాళీ
→ §11 · అసలు ఉపయోగమే లేదు

"O(1)" అనేది ఒక షరతులతో కూడిన వాగ్దానం

పుస్తకాల్లో "HashMap lookup is O(1)" అని ఉంటుంది. నిజం: **సగటున O(1)**.

చెత్త పరిస్థితిలో అది **O(n)** – అన్ని keys ఒకే bucket lo పడితే.

ఈ doc మొత్తం ఆ "సగటున" ని **ఎలా కాపాడాలి** అనే దాని గురించే. అదే interview ప్రశ్న కూడా.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Keys ఏ రకం – strings మాత్రమేనా, ఏదైనానా?	Strings అయితే ఒక string hash. Objects అయితే **hashCode/equals ఒప్పందం** (§12)
Collisions ని ఎలా – chaining నా open addressing నా?	ఇది ఒక నిజమైన ఎంపిక; §16 lo trade-offs
ఎన్ని entries ఊహించాలి?	ప్రారంభ capacity ని నిర్ణయిస్తుంది – resizes తగ్గుతాయి
Iteration క్రమం ముఖ్యమా?	ముఖ్యమైతే insertion order ని విడిగా ఉంచాలి (JS `Map` అదే చేస్తుంది)
`null` key అనుమతించాలా?	Java HashMap ఒక null key అనుమతిస్తుంది; ఇది ఒక API ఎంపిక
Thread-safe కావాలా?	ఇది **మరో లేవనెత్తాలి** – resize మధ్యలో చదివితే? (§16)
Delete తరచుగా జరుగుతుందా?	Chaining lo సులభం; open addressing lo **tombstones** కావాలి

"KEYS ఏ రకం?" – ఇది అనిపించినంత సాధారణమైనది కాదు

Strings అయితే సులభం – string నుంచి ఒక సంఖ్య లెక్కించడం స్పష్టం.

కానీ **objects** keys అయితే – రెండు ప్రశ్నలు వస్తాయి: (1) ఒక object నుంచి సంఖ్య ఎలా? (2) రెండు objects "ఒకటే" అని ఎలా తెలుసుకోవాలి – అదే reference నా, అదే విలువలా?

Java lo ఇదే `hashCode()` / `equals()` ఒప్పందం, మరియు దాన్ని ఉల్లంఘిస్తే HashMap నిశ్శబ్దంగా తప్పుగా ప్రవర్తిస్తుంది. §12 lo దాన్ని కళ్ళతో చూద్దాం.

Part 2 – మొదటి విరుపు: వరుస వెతుకులాట

---

3. Step – ఒక array, ఒక loop

Hash గురించి ఇంకా ఆలోచించొద్దు. అతి సులభమైన "key → value" ఏమిటి? **ఒక జాబితా, మరియు వెతకడం.**

```
class ArrayMap {
  constructor() { this.entries = []; }

  set(key, value) {
    const e = this.entries.find((x) => x.key === key);       // ← వరుస వెతుకులాట
    if (e) { e.value = value; return; }                       // ఉన్నదాన్ని మార్చడం
    this.entries.push({ key, value });
  }

  get(key) {
    const e = this.entries.find((x) => x.key === key);       // ← మళ్ళీ వరుస వెతుకులాట
    return e ? e.value : undefined;
  }
}
```

```
name  → Ravi
city  → Hyderabad
email → undefined
size  → 2 (name రెండుసార్లు కాదు)
```

సరిగ్గా పనిచేస్తోంది. `name` ని రెండుసార్లు set చేస్తే అది *మారింది*, కొత్తది చేరలేదు. లేని key కి `undefined`. API పరంగా ఇది ఒక నిజమైన Map.

మరి సమస్య ఏమిటి?

---

4. మొదటి విరుపు – 10 ms నుంచి 78 ms కి

`find` ప్రతిసారీ **మొదటి నుంచి** వెతుకుతుంది. Entries పెరిగితే ఏమవుతుందో కొలుద్దాం – ప్రతిసారీ 5,000 lookups, మొత్తం array అంతటా పంచి:

```
for (const n of [1000, 5000, 20000]) {
  const m = new ArrayMap();
  for (let i = 0; i < n; i++) m.set('k' + i, i);
```

```
  // మొత్తం array అంతటా వెతకాలి — ముందువి మాత్రమే కాదు
  const probes = Array.from({ length: 5000 }, (_, i) => 'k' + Math.floor(i * n /
5000));
  const t = process.hrtime.bigint();
  for (const p of probes) m.get(p);
  console.log(`${n} entries → 5,000 lookups: ${(Number(process.hrtime.bigint()-
t)/1e6).toFixed(0)} ms`);
}
```

```
  1000 entries → 5,000 lookups: 10 ms
  5000 entries → 5,000 lookups: 24 ms
 20000 entries → 5,000 lookups: 78 ms
```

ఈ timings ఒక నిజమైన run నుంచి; ప్రతి run lo ±20% తేడా వస్తుంది. **ఖచ్చితమైన సంఖ్య కాదు, వాలు ముఖ్యం** – entries పెరిగితే సమయం పెరుగుతోంది, మరియు అది ప్రతి run lo నిజం.

మొదటి విరుపు – ఖర్చు పరిమాణంతో పాటు పెరుగుతోంది

Entries 20 రెట్లు పెరిగితే సమయం దాదాపు 8 రెట్లు పెరిగింది. ఇది **O(n)** – సరిగ్గా HashMap కాకూడని లక్షణం.

ఇక్కడ ఒక పాఠం ఉంది, మరియు అది benchmark రాయడం గురించి: నా మొదటి కొలతలో నేను `m.get('k' + (i % n))` అని రాశాను. n = 20,000 కి అది కేవలం k0-k4999 ని మాత్రమే వెతికింది – అవన్నీ array **ముందు భాగంలో** ఉన్నాయి, కాబట్టి `find` త్వరగా తిరిగి వచ్చింది, మరియు సమయం *పెరగలేదు*.

తప్పు benchmark తప్పు నిర్ధారణ ఇస్తుంది. Probes ని మొత్తం పరిధిలో పంచాక నిజం కనిపించింది. Interview lo "I'd measure it" అనేది మంచి మాట – కానీ *సరిగ్గా* కొలవడం ఒక నైపుణ్యం.

ఏమి కావాలి

మనకి కావలసినది: **key ని చూసి, అది ఎక్కడ ఉందో నేరుగా లెక్కించడం.** వెతకకుండా.

అంటే ఒక function కావాలి: `key → index`. అదే **hash function**.

Part 3 – రెండో విరుపు: ఢీకొన్న keys

5. Step – Key ని ఒక చోటిగా మార్చడం

ఒక string ని ఒక సంఖ్యగా మార్చే function. **djb2** ఒక క్లాసిక్ – సులభం, మరియు బాగా పంచుతుంది:

```
function hash(key) {
  let h = 5381;
  for (let i = 0; i < key.length; i++)
    h = ((h << 5) + h + key.charCodeAt(i)) | 0;       // h * 33 + charCode
  return h >>> 0;                                      // ఋణాత్మకం కాకుండా
}
```

`(h << 5) + h` అంటే `h * 33`. ఆ 33 ఒక మంతిరం కాదు – చిన్న bit నమూనాలని బాగా కలిపే ఒక **బేసి సంఖ్య**. `| 0` ప్రతి అడుగులోనూ 32-bit కి కుదిస్తుంది (overflow ఆపడానికి), `>>> 0` చివర్లో దాన్ని ధనాత్మకం చేస్తుంది.

ఇప్పుడు ఆ సంఖ్యని ఒక index గా మార్చడం – `% size`:

```
class SlotMap {
  constructor(capacity = 8) { this.slots = new Array(capacity).fill(null); }
  #index(key) { return hash(key) % this.slots.length; }
  set(key, value) { this.slots[this.#index(key)] = { key, value }; }     // ← నేరుగా
  get(key) { const s = this.slots[this.#index(key)]; return s ? s.value : undefined;
}
}
```

వెతుకులాట పూర్తిగా పోయింది. `set` మరియు `get` రెండూ ఒక్క లెక్క, ఒక్క array access. ఇదే మనం కోరుకున్న O(1).

6. రెండో విరుపు – `city` వచ్చి `name` ని తినేసింది

మన మొదటి ఉదాహరణనే నడుపుదాం – రెండు keys, అంతే:

```
const m = new SlotMap(8);
m.set('name', 'Asha');
m.set('city', 'Hyderabad');
console.log('name →', m.get('name'), '| city →', m.get('city'));
```

```
name → Hyderabad | city → Hyderabad
slots: 6:city
```

రెండో విరుపు – ఇది మన మొదటి ఉదాహరణలోనే జరిగింది

`m.get('name')` **"Hyderabad"** ఇచ్చింది. "Asha" ఎక్కడా లేదు. `slots` చూస్తే ఒకే ఒక entry – `6: city`.

ఎందుకు? hash విలువలు చూడండి:

`name hash=2090536006 slot=6`
`city hash=2090149662 slot=6`

రెండు వేర్వేరు hash విలువలు, కానీ **% 8 తర్వాత రెండూ 6**. కాబట్టి `city` వచ్చినప్పుడు అది `name` మీదే రాసేసింది.

ఇది ఒక అరుదైన దురదృష్టం కాదు. ఇది **అనివార్యం**: hash విలువలు 4 బిలియన్ ఉంటాయి, slots 8 మాత్రమే. 4 బిలియన్ ని 8 గదుల్లో పెడితే ఢీకొట్టడం తప్పదు. **ఏ hash function కూడా దీన్ని ఆపలేదు.**

Pigeonhole సూత్రం

ఇది ఒక గణిత వాస్తవం: **9 పావురాలని 8 గూళ్ళలో పెడితే, ఏదో ఒక గూట్లో రెండు ఉంటాయి.** Keys slots కంటే ఎక్కువ ఉంటే ఢీకొట్టడం **తప్పనిసరి**.

కాబట్టి సరైన ప్రశ్న "ఎలా ఆపాలి?" కాదు – **"ఢీకొట్టినప్పుడు ఏం చేయాలి?"**

---

7. Step – Chaining

అతి సులభమైన జవాబు: **ప్రతి slot ఒక విలువ కాదు, ఒక జాబితా.** ఒకే slot కి పలు keys వస్తే, అవన్నీ ఆ జాబితాలో కూర్చుంటాయి.

```
class ChainMap {
  #buckets; #count = 0;
  constructor(capacity = 8) {
    this.#buckets = Array.from({ length: capacity }, () => []);   // ప్రతిదీ ఒక ఖాళీ
జాబితా
  }
  #index(key) { return hash(key) % this.#buckets.length; }

  set(key, value) {
    const b = this.#buckets[this.#index(key)];
    const e = b.find((x) => x.key === key);        // ఆ *చిన్న* జాబితాలో మాత్రమే వెతకడం
    if (e) { e.value = value; return; }
    b.push({ key, value }); this.#count++;
  }
  get(key) {
    const e = this.#buckets[this.#index(key)].find((x) => x.key === key);
    return e ? e.value : undefined;
```

```
  }
}
```

```
name → Asha | city → Hyderabad   ← రెండూ సరిగ్గా
buckets: 6:[name,city]
```

రెండూ బతికాయి, మరియు అవి రెండూ bucket 6 lo ఉన్నాయని స్పష్టంగా కనిపిస్తోంది.

ఇక్కడ `find` మళ్ళీ వచ్చింది – §3 lo మనం తొలగించాలనుకున్నదే. కానీ **ఇప్పుడు అది మొత్తం map మీద కాదు, ఒక bucket మీద మాత్రమే**. 10,000 entries మరియు 10,000 buckets ఉంటే, ఒక్కో bucket lo సగటున ఒకే entry – కాబట్టి ఆ `find` ఒక్క పోలిక.

అంటే **O(1) అనేది నిజానికి "buckets సరిపడా ఉంటే O(1)"**. ఆ షరత విరిగితే ఏమవుతుంది? – అదే తర్వాతి Part.

Part 4 – మూడో విరుపు: పొడవైపోతున్న గొలుసులు

---

8. మూడో విరుపు – 1,122 పొడవు గల ఒక గొలుసు

Buckets 16 వద్దే ఉంచి, entries పెంచుదాం. ప్రతిసారీ 50,000 lookups:

```
16 buckets, స్థిరం · 50,000 lookups:
     160 entries · load   10 · పొడవైన chain    16 → 4 ms
    1600 entries · load  100 · పొడవైన chain   126 → 8 ms
   16000 entries · load 1000 · పొడవైన chain  1122 → 43 ms
```

మూడో విరుపు – HASHMAP నెమ్మదిగా ఒక జాబితాగా మారుతోంది

16,000 entries, 16 buckets. అంటే **ఒక్కో bucket lo సగటున వెయ్యి entries**, మరియు అత్యంత పొడవైనది **1,122**.

ఆ bucket lo ఒక key ని వెతకాలంటే – 1,122 పోలికలు. అది O(1) కాదు, **అది O(n)**. మనం §4 lo తప్పించుకున్న సమస్యకే తిరిగి వచ్చాం, కేవలం 16 రెట్లు మెరుగైన రూపంలో.

మౌలిక తప్పు: buckets సంఖ్యని **స్థిరంగా** ఉంచాం. Entries పెరుగుతున్నప్పుడు buckets కూడా పెరగాలి.

Load factor

"ఎంత నిండింది?" అని చెప్పే ఒక సంఖ్య కావాలి:

> **load factor = entries ÷ buckets**

- `0.5` అంటే – రెండు buckets కి ఒక entry. చాలా ఖాళీ, కానీ వేగం.
- `1.0` అంటే – సగటున ఒక్కో bucket కి ఒక entry. మంచి సమతుల్యత.
- `1000` అంటే – పైన చూసిన విపత్తు.

కాబట్టి నియమం: **load factor ఒక హద్దు దాటితే, buckets ని పెంచడం.**

---

9. Step – Resize మరియు rehash

`set` lo, entry చేర్చాక load factor తనిఖీ చేసి, అవసరమైతే పెంచడం:

```
#grow() {
  const old = this.#buckets;
  this.#buckets = Array.from({ length: old.length * 2 }, () => []);   // రెట్టింపు
  for (const bucket of old)
    for (const e of bucket) this.#buckets[this.#index(e.key)].push(e);  // ← rehash
  this.#resizes++;
}

set(key, value) {
  const b = this.#buckets[this.#index(key)];
  const e = b.find((x) => x.key === key);
  if (e) { e.value = value; return; }
  b.push({ key, value }); this.#count++;
  if (this.#count / this.#buckets.length > this.loadFactor) this.#grow();
}
```

REHASH ని మర్చిపోవడం – ఇది ఒక క్లాసిక్ BUG

పాత entries ని కొత్త array కి **నేరుగా copy చేయకూడదు**. ఎందుకంటే index `hash(key) % size`, మరియు **size మారింది**.

ఒక key 16 buckets lo slot 6 lo ఉండొచ్చు, కానీ 32 buckets lo అది slot 22 కి చెందుతుంది. నేరుగా copy చేస్తే – అది slot 6 lo ఉండిపోతుంది, మరియు `get` slot 22 lo వెతుకుతుంది. **Entry map lo ఉంది, కానీ ఎప్పటికీ దొరకదు.**

అందుకే ప్రతి entry యొక్క index ని **మళ్ళీ లెక్కించాలి**. అదే "rehash" అనే పదం యొక్క అర్థం, మరియు అందుకే resize ఖరీదైనది.

అదే పరీక్ష, ఇప్పుడు resize తో:

```
resize తో · 50,000 lookups:
     160 entries ·    256 buckets · 4 resizes · పొడవైన chain 2 → 3 ms
    1600 entries ·   4096 buckets · 8 resizes · పొడవైన chain 2 → 2 ms
   16000 entries ·  32768 buckets · 11 resizes · పొడవైన chain 4 → 1 ms
```

అదే డేటా, RESIZE తో మరియు లేకుండా

16,000 entries · 50,000 lookups

16 buckets, స్థిరం	Resize తో (load 0.75)
పొడవైన chain: **1,122**	పొడవైన chain: **4**
సమయం: **43 ms**	సమయం: **1 ms**
ఇది ఒక జాబితా, ఒక map కాదు	32,768 buckets · 11 resizes

ఇక్కడ ముఖ్యమైనది వేగం కాదు – పొడవైన chain

160 entries → chain 2 · 1,600 → chain 2 · 16,000 → chain **4**

Entries 100 రెట్లు పెరిగినా, చెత్త పరిస్థితిలో కూడా 4 పోలికలే.

అదే O(1) యొక్క నిజమైన అర్థం – "వేగం" కాదు, "పరిమాణంతో పెరగకపోవడం".

10. Load factor 0.75 ఎందుకు · రెట్టింపు ఎందుకు

ఈ రెండు సంఖ్యలూ interview lo అడిగే ప్రశ్నలు, మరియు రెండిటికీ నిజమైన కారణాలు ఉన్నాయి.

ఎందుకు 0.75?

ఇది **సమయం vs స్థలం** మధ్య ఒక రాజీ:

Load factor	Chains	Memory	పరిణామం
0.5	చాలా చిన్నవి	సగం buckets ఖాళీ	వేగం, కానీ వృథా
0.75	చిన్నవి	సమతుల్యం	**Java, Python డిఫాల్ట్**
1.0	మధ్యస్థం	తక్కువ వృథా	ఢీకొట్టడం గమనించదగినంత
2.0+	పొడవైనవి	అతి తక్కువ	O(1) వాగ్దానం బలహీనం

0.75 ఏదో ఒక ప్రయోగం నుంచి వచ్చిన సంఖ్య – ఆ స్థాయిలో ఢీకొట్టడం ఇంకా అరుదు, మరియు memory వృథా ఆమోదయోగ్యం. **ఇది ఒక సూత్రం కాదు, ఒక అనుభవపూర్వక ఎంపిక**, మరియు అలా చెప్పడమే సరైన జవాబు.

ఎందుకు రెట్టింపు, +1 కాదు?

ఇది ఎక్కువ ఆసక్తికరం. Resize **ఖరీదైనది** – ప్రతి entry నీ rehash చేయాలి, అంటే O(n).

- **+1 చొప్పున పెంచితే:** n entries చేర్చడానికి n resizes, ప్రతిదీ O(n) → మొత్తం **$O(n^2)$**. 10,000 entries కి అది 100 మిలియన్ operations.
- **రెట్టింపు చేస్తే:** resizes కేవలం **$\log_2(n)$** సార్లు – పై కొలతలో 16,000 entries కి **11 resizes** మాత్రమే. మరియు మొత్తం rehash ఖర్చు `n + n/2 + n/4 + … < 2n` – అంటే **O(n)**.

> అంటే ఒక్కో `set` యొక్క **సగటు (amortised)** ఖర్చు O(1) గానే ఉంటుంది, అప్పుడప్పుడూ ఒక ఖరీదైన resize వచ్చినా. **Interview lo "amortised O(1)" అనే పదం వాడండి** – ఇది మీరు resize ఖర్చు గురించి ఆలోచించారని చూపిస్తుంది.

Part 5 – Hash function యొక్క నాణ్యత

11. చెడ్డ hash – 256 buckets, 4 మాత్రమే వాడబడ్డాయి

ఇప్పటిదాకా మనం hash function *మంచిది* అని ఊహించాం. అది చెడ్డదైతే?

ఒక "సహజమైన" కానీ చెడ్డ hash: **key యొక్క పొడవు**.

```
function badHash(key) { return key.length; }      // ← పొడవు మాత్రమే!
```

2,000 keys (`user_0` నుంచి `user_1999`) – అన్నీ 6-10 అక్షరాలు. 256 buckets:

```
మంచి hash : { usedBuckets: 256, empty: 0, longestChain: 11 }
చెడ్డ hash: { usedBuckets: 4, empty: 252, longestChain: 1000 }
```

చెడ్డ HASH – 252 BUCKETS ఖాళీగా కూర్చున్నాయి

2,000 keys, 256 buckets – కానీ **4 buckets మాత్రమే** వాడబడ్డాయి, మరియు ఒక్క bucket lo **1,000 entries**.

మీ resize logic ఖచ్చితంగా ఉండొచ్చు, load factor సరిగ్గా ఉండొచ్చు – **ఏమీ ఉపయోగం లేదు**. Resize buckets ని రెట్టింపు చేస్తుంది, కానీ చెడ్డ hash వాటిని ఎలాగూ వాడదు. మీ HashMap ఒక linked list.

మంచి hash function కి రెండు షరతులు: (1) ఒకే key ఎప్పుడూ ఒకే సంఖ్య ఇవ్వాలి – *ఖచ్చితత్వం*; (2) వేర్వేరు keys buckets అంతటా **సమానంగా** పంచాలి – *పంపిణీ*. రెండోది లేకపోతే మిగతాదంతా వృథా.

12. మారే key – ఒక entry ని శాశ్వతంగా పోగొట్టుకోవడం

Keys objects అయితే ఒక సూక్ష్మమైన, మరియు చాలా ప్రమాదకరమైన ఉచ్చు ఉంది.

```
const user = { id: 42, name: 'Asha' };
m.set(user, 'Asha యొక్క profile');
console.log('చేర్చిన వెంటనే  :', m.get(user));

user.id = 99;                                    // ← key ని మార్చాం
console.log('id మార్చాక     :', m.get(user), '  ← అదే object!');
```

```
చేర్చిన వెంటనే  : Asha యొక్క profile
id మార్చాక     : undefined   ← అదే object!
=> object map lo ఉంది, కానీ దాన్ని ఇక ఎప్పటికీ వెతకలేం.
```

ENTRY పోలేదు – కానీ దానికి దారి పోయింది

మనం **అదే object** ని పాస్ చేశాం, మరియు `undefined` వచ్చింది.

కారణం: `set` సమయంలో `hash({id:42})` ఒక bucket ఇచ్చింది, మరియు entry అక్కడ కూర్చుంది. ఆపై `id` మారింది, కాబట్టి `get` ఇప్పుడు **వేరే bucket** ని లెక్కిస్తోంది. Entry ఇంకా పాత bucket lo ఉంది – memory తింటూ, ఎవరికీ అందకుండా.

నియమం: map lo key గా వాడే ఏ object నీ, అది map lo ఉన్నంతకాలం **మార్చకూడదు**. అందుకే Java lo `String` immutable – మరియు అందుకే అది అత్యంత సాధారణ HashMap key.

Interview lo ఇది చెప్పడం ఒక బలమైన సంకేతం, ఎందుకంటే ఇది మీరు hashCode/equals ఒప్పందాన్ని నిజంగా అర్థం చేసుకున్నారని చూపిస్తుంది – కేవలం అప్పజెప్పట్లేదని.

Part 6 — పూర్తి system

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

```
function hashString(str) {                       // djb2
  let h = 5381;
  for (let i = 0; i < str.length; i++) h = ((h << 5) + h + str.charCodeAt(i)) | 0;
  return h >>> 0;
}

class HashMap {
  #buckets; #size = 0; #resizes = 0;

  constructor(capacity = 16, { loadFactor = 0.75, hash = hashString } = {}) {
    if (capacity < 1) throw new Error('CAPACITY_MUST_BE_POSITIVE');
    this.#buckets = Array.from({ length: capacity }, () => []);
    Object.assign(this, { loadFactor, hash });   // hash ని inject చేయడం — test కి,
tuning కి
  }

  #indexFor(key, len = this.#buckets.length) { return this.hash(String(key)) % len; }

  #grow() {
    const old = this.#buckets;
    this.#buckets = Array.from({ length: old.length * 2 }, () => []);
    for (const bucket of old)
      for (const e of bucket) this.#buckets[this.#indexFor(e.key)].push(e);   //
rehash
    this.#resizes++;
  }

  set(key, value) {
    const b = this.#buckets[this.#indexFor(key)];
    const e = b.find((x) => x.key === key);
    if (e) { e.value = value; return this; }                 // ఉన్నదాన్ని మార్చడం
    b.push({ key, value }); this.#size++;
    if (this.#size / this.#buckets.length > this.loadFactor) this.#grow();
    return this;                                             // chaining కోసం
  }

  get(key) {
    const e = this.#buckets[this.#indexFor(key)].find((x) => x.key === key);
    return e ? e.value : undefined;
  }

  has(key) { return this.#buckets[this.#indexFor(key)].some((x) => x.key === key); }

  delete(key) {
    const b = this.#buckets[this.#indexFor(key)];
    const i = b.findIndex((x) => x.key === key);
    if (i === -1) return false;
```

```
    b.splice(i, 1); this.#size--;                 // chaining lo tombstone అవసరం లేదు
    return true;
  }

  *entries() { for (const b of this.#buckets) for (const e of b) yield [e.key,
e.value]; }
  keys() { return [...this.entries()].map(([k]) => k); }
  get size() { return this.#size; }

  get stats() {
    const lens = this.#buckets.map((b) => b.length);
    return { entries: this.#size, buckets: this.#buckets.length,
             loadFactor: +(this.#size / this.#buckets.length).toFixed(2),
             resizes: this.#resizes, usedBuckets: lens.filter((l) => l > 0).length,
             longestChain: Math.max(...lens) };
  }
}
```

```
name    → Asha
city    → Hyderabad   ← §4 lo ఇది "Asha" ని తినేసింది
missing → undefined
has(role)→ true | has(x) → false

name మార్చాక → Ravi | size 3 ← పెరగలేదు
delete(city) → true | size 2
delete(city) మళ్ళీ → false
keys: [ 'name', 'role' ]

--- 10,000 entries ---
{
  entries: 10000,
  buckets: 16384,
  loadFactor: 0.61,
  resizes: 10,
  usedBuckets: 6389,
  longestChain: 4
}
user_9999 → 9999
delete తర్వాత size: 9999 | user_5000 → undefined
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`name → Asha`, `city → Hyderabad` – §6 lo `city` వచ్చి `name` ని తినేసింది. ఇప్పుడు రెండూ బతికాయి, అదే 8-bucket map lo.

`size 3` (name మార్చాక) – ఉన్న key ని మళ్ళీ `set` చేస్తే size పెరగదు. ఇది సరైన Map ప్రవర్తన.

`delete(city)` రెండోసారి `false` – లేని దాన్ని తీసేయడం ఒక తప్పు కాదు, కానీ అది `false` చెప్పాలి. Caller కి తేడా తెలియాలి.

10,000 entries: `buckets: 16384`, `resizes: 10`. 16 నుంచి మొదలై పది రెట్టింపులు = 16,384. `log₂(16384/16) = 10` – §10 lo చెప్పిన సిద్ధాంతం సరిగ్గా సరిపోయింది.

`usedBuckets: 6389` **of 16,384.** అంటే దాదాపు 39% buckets వాడబడ్డాయి. ఇది తక్కువగా అనిపిస్తుంది కానీ ఇది **యాదృచ్ఛిక పంపిణీకి ఆశించినదే** – load factor 0.61 వద్ద, గణితం ప్రకారం `1 − e^(−0.61) ≈ 46%` దగ్గరగా. §11 lo చూసిన **చెడ్డ hash యొక్క 1.5%** తో దీన్ని పోల్చి చూడండి.

`longestChain: 4`. 10,000 entries, మరియు చెత్త పరిస్థితిలో 4 పోలికలు. **అదే O(1).**

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	`set`, `get` API	మౌలిక ఒప్పందం
§4 (విరుపు)	`hash()` + `% size`	వరుస వెతుకులాట O(n)
§6 (విరుపు)	Buckets = జాబితాలు	ఢీకొట్టడం ఒక విలువని తినేసింది
§8 (విరుపు)	`loadFactor`, `#grow()`	Chain 1,122 కి పెరిగింది
§9	Rehash (copy కాదు)	Size మారితే index మారుతుంది
§11	`hash` ని inject చేయడం	Hash నాణ్యతని పరీక్షించడానికి
ఇక్కడ	`delete`, `entries`, `stats`	పూర్తి Map API

Part 7 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Code 15 నిమిషాల్లో అయిపోతుంది – మిగతాదంతా "ఎందుకు"

4 నిమి	**Clarify** – key రకం, collisions, thread safety
6 నిమి	hash + % size · **ఢీకొట్టడం అనివార్యం** అని చూపించడం
11 నిమి – Chaining	set / get / delete · ఇది సులభ భాగం
13 నిమి – Resize	Rehash · **0.75 ఎందుకు · రెట్టింపు ఎందుకు**
11 నిమి	Hash నాణ్యత, mutable keys, thread safety

ఏమి తప్పక చెప్పాలి

1. **ఢీకొట్టడం అనివార్యం** (§6) – pigeonhole. "ఎలా ఆపాలి" కాదు, "ఏం చేయాలి".
2. **Resize lo rehash తప్పనిసరి** (§9) – నేరుగా copy చేస్తే entries అందుబాటులో ఉండవు.
3. **రెట్టింపు ఎందుకు** (§10) – +1 అయితే $O(n^2)$; రెట్టింపు అయితే amortised $O(1)$.
4. **Hash నాణ్యత** (§11) – చెడ్డ hash తో resize logic పనికిరాదు.
5. **Mutable keys** (§12) – ఇది చాలా తక్కువ మంది చెప్తారు.

ఏమి వదిలేయాలి

- `entries()`, `keys()` – స్పష్టం, సమయం వృథా.
- `stats()` – debugging కోసం, design కోసం కాదు.
- **djb2 ని వివరించొద్దు** – "a standard string hash" అని చెప్పి ముందుకి వెళ్ళండి.

15. నోటితో చెప్పాల్సిన English script

"A few questions first. What are the keys – strings, or arbitrary objects? Do I need thread safety? And roughly how many entries should I expect?

The core idea is to compute the location from the key instead of searching for it. A hash function turns the key into a number, and modulo the bucket count turns that into an index.

But collisions are unavoidable, and I want to be clear about why: the hash space is four billion values and I might have sixteen buckets. That's pigeonhole – you can't avoid it, so the question isn't how to prevent collisions, it's what to do when they happen. In my first two-key example, 'name' and 'city' both landed in bucket six, and the second one silently overwrote the first.

I'll handle it with chaining – each bucket holds a small list. Lookup hashes to the bucket and scans just that list. With a good spread that list has one or two entries, so it's still constant time.

That constant-time claim has a condition though: enough buckets. If I fix the count at sixteen and insert sixteen thousand entries, the longest chain gets to about eleven hundred and it's O(n) again – I measured that. So I track load factor, entries over buckets, and grow when it passes a threshold.

The critical detail on growing is that I can't just copy the old array over. The index is hash modulo size, and size changed – so every entry has to be rehashed into its new bucket. Copying directly leaves entries sitting in buckets where lookup will never look for them. They're in the map and permanently unreachable.

I double the capacity rather than growing by a fixed amount. Growing by one would mean a rehash on every insert, so n inserts cost $O(n^2)$. Doubling means log n resizes and the total rehash work is bounded by 2n, so each insert is amortised O(1).

Threshold-wise, 0.75 is what Java and Python use. It's not a theorem, it's an empirical balance – collisions are still rare there and the wasted space is acceptable.

Two things I'd call out. First, all of this depends on the hash spreading well. If I hash on string length, two thousand keys land in four buckets out of 256 and the resize logic is useless – it's a linked list with extra steps. Second, if keys are mutable objects and someone changes a field after insertion, the entry becomes unreachable, because you'd now hash to a different bucket. That's why immutable keys like strings are the norm."

16. Follow-ups – open addressing, thread safety, treeify

Follow-up	జవాబు	మారే classes
"Capacity ని 2 యొక్క ఘాతంగా ఉంచితే?"	`% size` ని `& (size-1)` తో మార్చొచ్చు – వేగం	`#indexFor`
"Iteration క్రమం insertion order కావాలి"	Entries కి ఒక doubly linked list – `LinkedHashMap`	+ఒక list
"Hash ని inject చేయగలగాలి"	ఇప్పటికే ఉంది (constructor lo)	**0**
"Shrink కూడా చేయాలా?"	Load చాలా తగ్గితే – కానీ జాగ్రత్త: grow/shrink హద్దులు వేరుగా ఉండాలి (thrashing)	`delete`
"Chaining కాకుండా?"	కింద చూడండి	–
"Thread-safe"	కింద చూడండి	–

Open addressing

> *"Chaining కాకుండా – ఢీకొన్నప్పుడు **తర్వాతి ఖాళీ slot** వెతకడం (linear probing). లాభం: అదనపు జాబితాలు లేవు, అంతా ఒకే array lo – cache కి చాలా మంచిది, మరియు memory తక్కువ.*
>
> ***ఖర్చు:** delete కష్టమవుతుంది. ఒక entry ని తీసేస్తే, దాని తర్వాత probe అయిన entries కి దారి తెగిపోతుంది. అందుకే ఒక **tombstone** పెట్టాలి – "ఇక్కడ ఏమీ లేదు, కానీ ముందుకి వెతుకు". Tombstones పేరుకుపోతే performance తగ్గుతుంది.*
>
> *అలాగే load factor **చాలా ముఖ్యం** అవుతుంది – 0.7 దాటాక linear probing వేగంగా దిగజారుతుంది, chaining అంత సున్నితంగా కాదు. **నేను chaining ఎంచుకున్నాను ఎందుకంటే delete సులభం మరియు ప్రవర్తన ఊహించదగినది.**"*

Thread safety

> *"రెండు threads ఒకేసారి `set` చేస్తే – ఒకే bucket lo రెండూ `push` చేయొచ్చు, మరియు ఒకటి పోవచ్చు. అంతకంటే ఘోరం: **ఒకరు resize చేస్తుండగా ఇంకొకరు చదివితే** – వాళ్ళు సగం పాత, సగం కొత్త array చూస్తారు.*
>
> *సులభ జవాబు: మొత్తం map కి ఒక lock. కానీ అప్పుడు అన్ని reads వరుసలో నిలబడతాయి.*
>
> ***నిజ systems ఏం చేస్తాయి:** Java యొక్క `ConcurrentHashMap` **bucket వారీగా** lock చేస్తుంది – వేర్వేరు buckets కి వేర్వేరు threads సమాంతరంగా రాయగలవు. ఇది Deep Dive 01 lo parking lot కి spot-వారీ lock లాంటిదే – **ఒకే ఆలోచన, వేరే domain.**"*

Java 8 యొక్క treeify

> *"చెడ్డ hash (లేదా ఒక దాడి) వల్ల ఒక bucket చాలా పొడవైతే – Java 8 ఆ chain ని ఒక* ***red-black tree*** *గా మారుస్తుంది (8 దాటితే). అప్పుడు ఆ bucket lo lookup O(n) కాకుండా* ***O(log n)*** *అవుతుంది.*
>
> *అంటే చెత్త పరిస్థితి O(n) నుంచి O(log n) కి మెరుగైంది. ఇది ఒక* ***hash collision DoS దాడి*** *కి రక్షణ – ఎవరైనా ఉద్దేశపూర్వకంగా ఒకే bucket కి పడే keys పంపితే."*

---

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
వెతకడం కాదు, లెక్కించడం	hash → index (§5)	Sharding, partitioning, CDN routing
ఢీకొట్టడం అనివార్యం	Pigeonhole (§6)	UUID, checksums, bloom filters
O(1) ఒక షరతులతో కూడిన వాగ్దానం	Load factor (§8)	ప్రతి "O(1)" వాదనలో
రెట్టింపు = amortised O(1)	Resize (§10)	Dynamic arrays, buffers, connection pools
అంచనా నాణ్యత మీదే అంతా ఆధారపడుతుంది	చెడ్డ hash (§11)	Load balancing, sampling, sketches
Key మారితే దారి పోతుంది	Mutable keys (§12)	Caching, memoization, DB indexes
సూక్ష్మ lock > స్థూల lock	Bucket-వారీ lock (§16)	Deep Dive 01 §9 తో అదే ఆలోచన

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot** · **02 LRU & LFU Cache** · **03 Rate Limiter** · **04 BookMyShow** · **05 Splitwise** · **06 Elevator** · **07 Pub-Sub** · **08 HashMap**.

ఈ doc ప్రత్యేకత: ఇది `LLD_Design_Problems_Telugu.pdf` lo **లేని** మొదటి problem – research ప్రకారం ఇది top-10 lo ఉండాల్సినది. Deep Dive 02 (Cache) lo మనం HashMap ని *వాడాం; ఇక్కడ దాన్ని కట్టాం.* ఆ రెండూ కలిపి చదివితే LRU cache యొక్క "HashMap + DLL" ఎందుకు అంత శక్తివంతమో పూర్తిగా అర్థమవుతుంది.

Data structures పునాది కోసం – `DSA_00_Foundations_Telugu.pdf`. Hashing problems కోసం – `DSA_03_Hashing_Intervals_Telugu.pdf`.

HashMap – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, timing, మరియు bucket పంపిణీ నిజంగా `node` lo run చేసి తీసినవే ✅